CodeSage

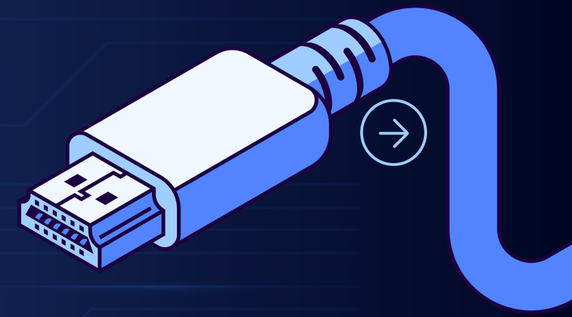
FIND. FIX. VERIFY.

codesage-theta.vercel.app



CodeSage

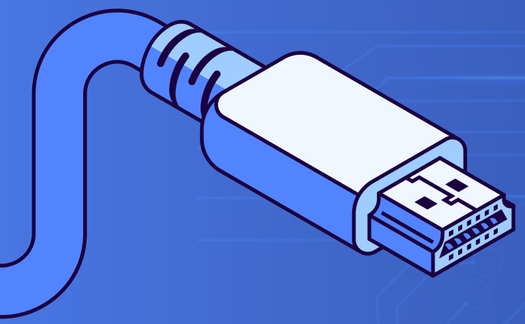
THE PROBLEM



AI can generate code. But can it verify a fix?

01. Limited repository understanding
AI-generated suggestions may lack sufficient context about the actual project structure, dependencies, function calls and existing implementation.

02. Unverified root-cause analysis
An AI can produce convincing explanations without inspecting the relevant source files or establishing evidence for its conclusions.



03. Manual implementation and validation
Developers still need to locate the affected code, implement suggested changes, inspect modifications and check whether anything else has broken.

04. Lack of execution-based verification
A generated patch may look correct but fail during execution, leave the original issue unresolved or introduce additional regressions.a

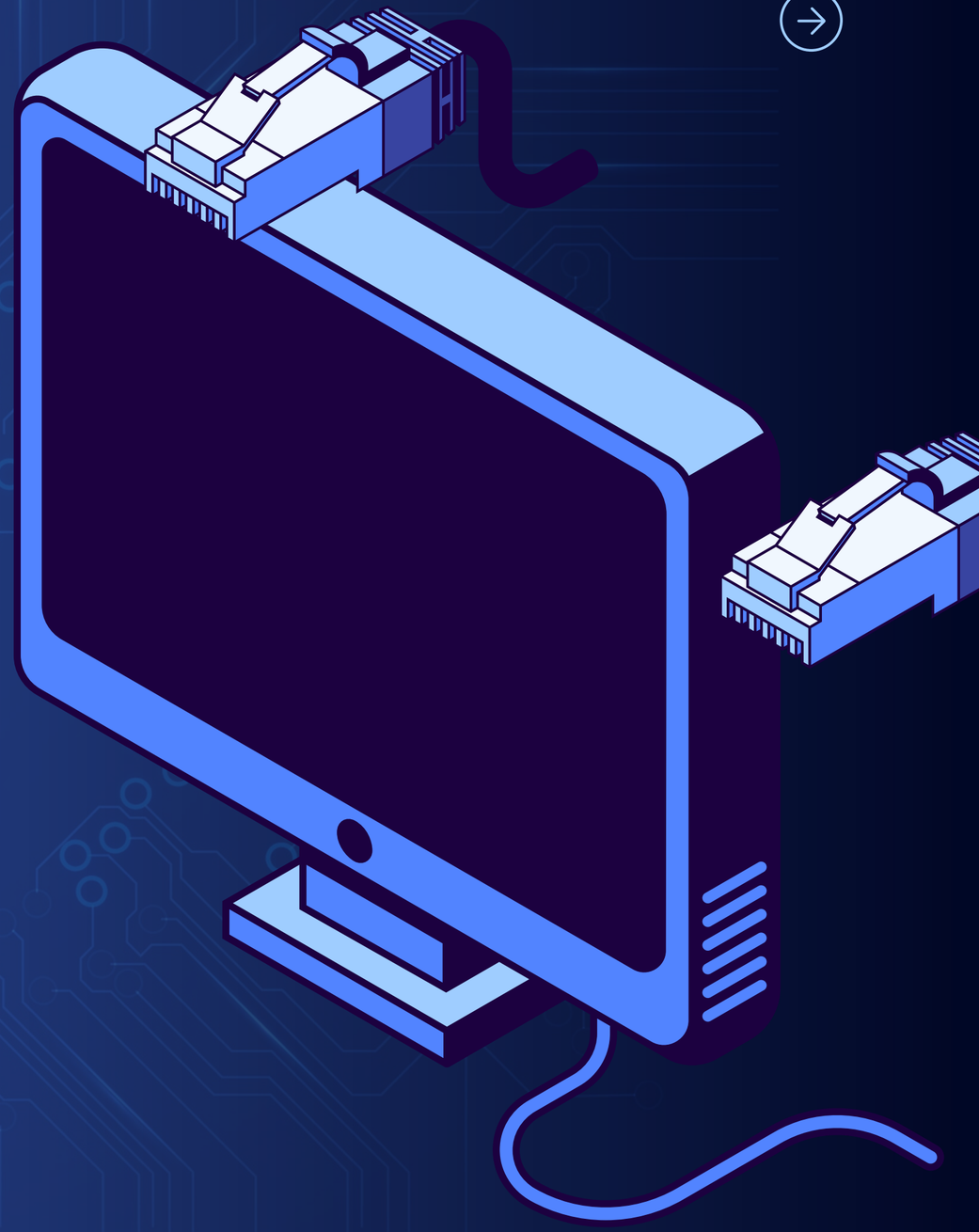
CodeSage

INTRODUCING CODESAGE

CodeSage is an autonomous AI debugging agent that connects repository understanding, evidence-based investigation, controlled code modification and automated verification into one engineering workflow.

Instead of generating a solution from a bug description alone, CodeSage interacts with the actual codebase to investigate the problem and produce an inspectable modification.

CodeSage combines repository-aware investigation, agentic debugging, automated patching and execution-based verification into one engineering workflow. It indexes the project, detects its technology stack and test commands, then uses a tool-calling agent to inspect source files, trace bugs and establish evidence-backed diagnoses. It generates unified-diff patches, applies them to an isolated repository copy and runs the project's test suite before and after the modification to determine the final outcome.



CodeSage



HOW CODESAGE **WORKS**

01 – Connect & Understand
Provide a local repository path or upload a project folder. CodeSage scans files, extracts code symbols, detects the technology stack and identifies the test command.

02 – Investigate & Diagnose
Extracts relevant keywords, searches and reads actual source files using AI tool calls, then identifies the root cause with supporting file:line evidence.

03 – Generate & Apply the Patch
Creates a unified-diff patch based on inspected code and applies it to an isolated repository copy, with an automatic correction attempt if patch application fails.

04 – Test, Verify & Report
Runs the project's test suite before and after the modification, compares actual execution results, computes the verdict and generates an engineering report with the evidence, patch and test results.

CodeSage

AGENTIC **INVESTIGATION**



CodeSage uses a multi-turn tool-calling agent to understand the actual implementation instead of relying exclusively on the reported symptoms.

How the investigation works:

- Extracts relevant keywords from the bug report.
- Searches the repository to identify candidate files.
- Uses search_code to locate relevant implementations.
- Uses read_file to inspect actual source code.
- Continues investigating using evidence from previous tool calls.
- Produces a structured diagnosis with root cause, affected files and source references.



CodeSage

PATCH ENGINEERING & ISOLATION



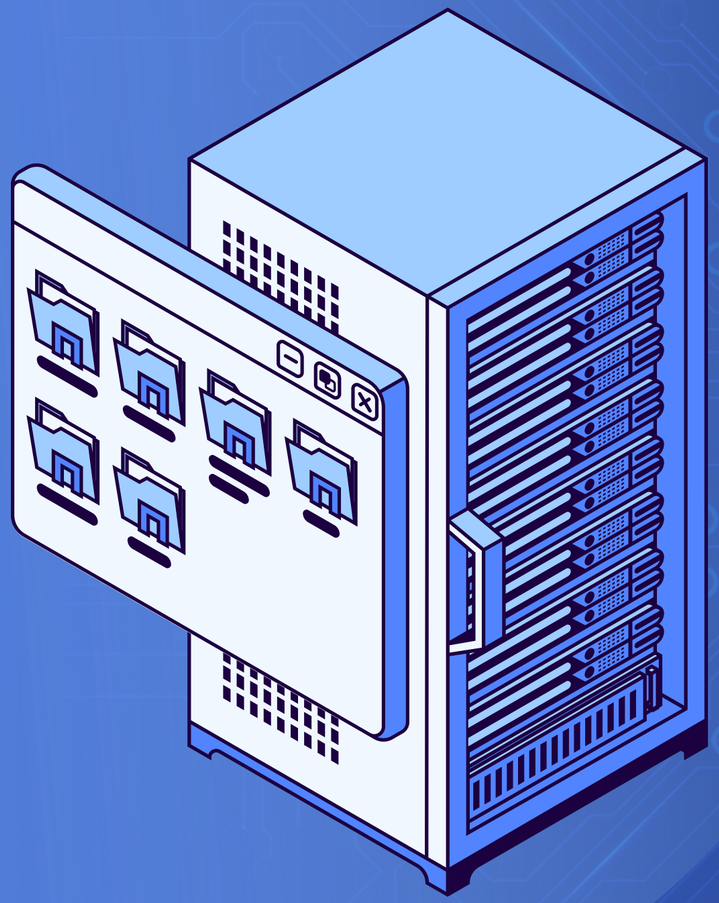
Patch application

- Generates diffs using the actual source code inspected by the agent.
- Repairs malformed diff hunk headers using file contents.
- Applies patches through git apply.
- Uses a text-patching fallback if Git application fails.
- Allows one correction retry using the actual application error.

Isolated workspace

- Creates a separate copy of the repository with its own local Git repository.
- Applies all modifications inside that copy.
- Preserves the original project without directly modifying it.
- Retains the patched workspace for file inspection and ZIP download.

CodeSage



MULTI-BUG ORCHESTRATION

- Structural bug splitting 01.
- Sequential investigation 02.
- Regression testing 03.
- Failure isolation 04.
- Consolidated reporting 05.

CodeSage

VERIFICATION **ENGINE**

A fix is only as credible as its test results.
CodeSage independently evaluates modifications using the project's actual test suite.

Before the patch 01.

- Detects and executes the test command
- Records actual pass/fail results and exit codes.

After the patch 02.

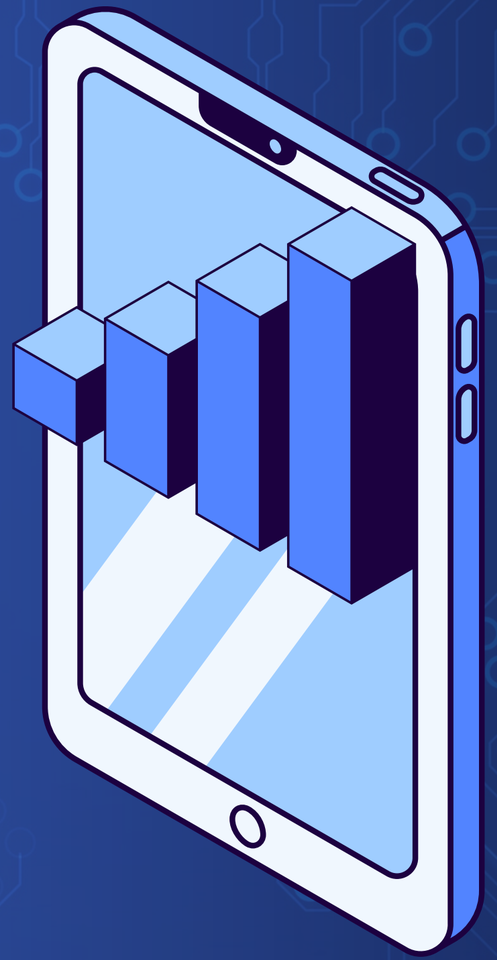
- Executes the test command again.
- Compares the results.
- Determines the final verification state.

CodeSage

TECHNICAL FOUNDATION

Frontend 01.

- React, Vite, Tailwind CSS
- Three-pane investigation console
- Evidence, verdict and diff viewer
- VS Code-style patched-workspace browser



Backend 02.

- Express and TypeScript
- REST APIs and asynchronous jobs
- Disk-persisted job state
- Folder uploads and workspace management

Core Pipeline 03.

- Repository indexing and code search
- Agentic investigation and tool calling
- Unified-diff patch engine
- Before/after test runner
- Multi-bug orchestration and report generation

CodeSage

VALIDATION & CLOSING



- Repository-level AI investigation.
- Evidence-backed root-cause analysis.
- Actual code modifications in an isolated workspace.
- Execution-based verification.
- Coordinated multi-bug debugging.

42
Unit tests passed
Core pipeline

4
Test projects
End-to-end validation

3
LLM providers
Integrated backends


CodeSage
THANK
YOU.
codesage-theta.vercel.app